

The Three-Dimensional Packing - Capacitated Profitable Tour Problem

(Authors' names are not included for peer review)

Authors are encouraged to submit new papers to INFORMS journals by means of a style file template, which includes the journal title. However, use of a template does not certify that the paper has been accepted for publication in the named journal. INFORMS journal templates are for the exclusive purpose of submitting to an INFORMS journal and are not intended to be a true representation of the article's final published form. Use of this template to distribute papers in print or online or to submit papers to another non-INFORM publication is prohibited.

Abstract. Medical drone delivery requires jointly determining which requests to serve, the order in which to visit them, and how to pack indivisible, temperature-sensitive cargo such as blood, vaccines, and diagnostic samples. In the literature, routing problems with profits explicitly consider customer selection but typically represent vehicle capacity using aggregate measures such as weight or volume. In contrast, routing problems with packing constraints explicitly account for the spatial arrangement of items within vehicles but generally assume that all customers must be served. However, the joint integration of customer selection, routing, and packing remains largely unexplored.

Motivated by this gap, this paper studies the Three-Dimensional Packing Capacitated Profitable Tour Problem (3DP-CPTP), which requires simultaneously selecting requests, determining vehicle routes, and packing items of different lengths, widths, heights, and weights. We formulate the 3DP-CPTP as a Mixed Integer Linear Program (MILP) and develop a matheuristic that embeds MILP components within a heuristic search framework. We further compare the matheuristic with Tabu Search (TS) and Simulated Annealing (SA) for solving the 3DP-CPTP. Our computational study provides a comprehensive evaluation on established instances with up to 100 customer request locations, 200 items, and 28 vehicles. SA achieves the best average performance across different computational time budgets, although TS outperforms SA in some individual scenarios.

Key words: Three-Dimensional packing, Capacitated profitable tour problem, Matheuristic, Tabu search, Simulated annealing

1. Introduction

Unmanned aerial vehicles, commonly known as drones, have become operational infrastructure in medical logistics rather than a technological novelty. In Rwanda, on-demand drone delivery has compressed blood-product delivery times from several hours by road to roughly fifteen minutes, and the national blood service now distributes the majority of the country's supply outside the capital by air (Jeon et al. 2022). Drones bypass impassable terrain and damaged road networks, and they

preserve the cold chain in settings where between a quarter and two-fifths of temperature-sensitive supplies are otherwise lost to inconsistent refrigeration.

The COVID-19 immunization campaign also demonstrated that logistics can be constrained not only by distance but also by physical packing requirements. In remote Southeast Asia, landslide exposure and severe terrain slope often make ground delivery unreliable, making aerial supply the only option (Adwibowo 2021). Most of the mass is thermal rather than pharmaceutical, since passive packing requires coolant at roughly 2.5 g per gram of vaccine and diluent, and the resulting containers are rigid, sealed, and indivisible. On a drone, their arrangement is not a packing convenience but a flight-feasibility condition, as routing and packing should not be decided sequentially, since a distance-optimal tour may admit no feasible packing.

Recently, medical drone delivery has also gained increasing attention as the German Luftfahrt-Bundesamt granted what has been described as Europe's first permit for the routine aerial delivery of medications over populated areas, allowing pharmacies to supply healthcare facilities via autonomous drones (Höhn 2026). In addition, at Klinikum Ingolstadt, whose central pharmacy supplies thirteen regional hospitals, drone delivery is expected to reduce transport times to approximately one-third of those required by road transport Zharinova (2026). Participating hospitals place requests through a mobile application, and drones can receive their next delivery assignment while still in flight, enabling the transport of both medications and laboratory samples.

The vast literature addresses routing and packing decisions separately. *Three-Dimensional Bin Packing* (3D-BPP) distributes packages into as few vehicles as possible subject to physical constraints on packages and vehicles. Three-dimensional *loading* problems additionally incorporate transportation-specific requirements such as cargo stability, weight distribution, loading and unloading order, vehicle regulations, and restricted item orientations arising from handling or safety considerations (Crainic, Perboli, and Tadei 2008, Bortfeldt and Gehring 2006). Integrating such loading constraints with routing gives rise to the *Three-Dimensional Loading Capacitated Vehicle Routing Problem* (3L-CVRP), first studied by Gendreau et al. (2006), which has subsequently motivated extensive research on related problem variants and solution methods. In parallel, routing problems with profits relax the requirement to visit all customers, allowing the selection of a subset of customers based on the trade-off between an associated prize and routing costs (Feillet, Dejax, and Gendreau 2005, Archetti et al. 2009).

Neither literature stream alone comprises the setting above, as integrated routing and loading formulations inherit from the classical VRP the assumption that every request must be served.

Medical drone operations routinely violate this assumption, as a limited fleet may face more requests than can be served within a given planning horizon. Thus, operators must decide which requests to serve, prioritizing by clinical urgency or delivery value. Routing problems with profits explicitly consider customer selection but typically represent vehicle capacity through aggregate measures such as weight or volume, without modeling detailed spatial loading feasibility. In contrast, packing-constrained routing problems account for the spatial arrangement of items within the vehicle but often assume a predetermined set of customers. These two decisions are inherently interdependent: accepting a customer changes the set of items that must be loaded, which may affect the feasibility of the loading configuration and, consequently, the set of customers that can be served within a route. Selection determines which items must be packed, while packing feasibility in turn determines which requests can be profitably selected. This interdependence arises whenever indivisible cargo, limited vehicle capacity, and heterogeneous request values interact.

Motivated by this gap, we introduce the *Three-Dimensional Packing Capacitated Profitable Tour Problem* (3DP-CPTP), where selecting, packing, and routing decisions must be made simultaneously. To the best of our knowledge, customer selection has not previously been combined with three-dimensional loading constraints in a routing model. Jointly optimizing selection, sequencing, and loading can yield materially better plans than treating the decisions independently, but substantially increases computational complexity. Furthermore, the 3DP-CPTP incorporates two additional sources of flexibility. First, the individual three-dimensional items constituting a customer's demand can be assigned to different vehicles, allowing multiple vehicles to jointly deliver to a customer. This flexibility has been shown to improve solution quality in classical profitable tour problems (see Caspar, Schermer, and Wendt (2023)) and vehicle routing problems with three-dimensional loading constraints (see Bortfeldt and Yi (2018)). Second, the 3DP-CPTP permits customer demands to be served partially at the item level, where each customer demand consists of individual items, and the associated prize is collected for each item that is served. Consequently, a customer need not be served in its entirety, as individual items can be selected and served independently.

Exact approaches to integrated routing and loading have been proposed primarily for two-dimensional variants and for comparatively small three-dimensional instances (Ferreira et al. 2025, Tamke et al. 2025), while the difficulty of the integrated problem has directed most attention toward metaheuristics, with tabu search particularly prevalent (Bortfeldt and Yi 2020).

Our main contributions are as follows:

1. To the best of our knowledge, existing studies do not jointly consider customer selection, profit-based routing, and detailed three-dimensional loading decisions within a single optimization framework. We introduce the 3DP-CPTP, provide a problem definition, and a *Mixed Integer Linear Programming* (MILP) formulation.
2. Following a brief overview of recent related literature, we discuss current literature on algorithms for solving closely related problems.
3. We adapt a matheuristic and a tabu search that have been used for closely related problems to solve the 3DP-CPTP.
4. While *Simulated Annealing* (SA) has been successfully applied to related routing and packing problems, its application to the integrated 3DP-CPTP has, to the best of our knowledge, not been investigated. We therefore investigate simulated annealing as a solution approach for the 3DP-CPTP.
5. Computational experiments on the well-known Gendreau et al. (2006) instances are performed to evaluate the effectiveness of our different solution methods.

The rest of this paper is structured as follows. Section 2 discusses related literature for the 3DP-CPTP and current algorithms used for solving closely related problems. In Section 3, we formally define the 3DP-CPTP, formulate it as an MILP, and provide a matheuristic with different destroy strategies. Furthermore, we present a tabu search and simulated annealing with five different move operators for solving the 3DP-CPTP. Section 4 presents extensive computational experiments based on well-known instances to validate the effectiveness of our solution approaches. Finally, Section 5 summarizes our findings and discusses potential directions for future research.

2. Related Literature

In this section, the literature concerning the 3DP-CPTP and closely related problems is reviewed, and recently proposed solution methods to them are discussed.

2.1. Selection and Loading Problems

Considered in isolation from the routing decision, the loading component of the 3DP-CPTP corresponds to a knapsack-type cutting and packing problem in the typology of Wäscher, Haußner, and Schumann (2007), where a subset of items is selected and packed into a container of fixed dimensions to maximize the total prize collected. The three-dimensional variant, in which items are placed orthogonally and without overlap, was introduced by Gilmore and Gomory (1965) and has since been studied extensively, with a recent overview given by Feng et al. (2025).

2.2. Integrated Loading and Routing and Profitable Customer Selection

Research on the 3L-CVRP has undergone significant advancements since the work of Gendreau et al. (2006), which introduced the integration of vehicle routing and three-dimensional loading constraints. Subsequent studies have incorporated a wide range of practical loading and routing features, including time windows, heterogeneous fleets, and pickup-and-delivery operations. A comprehensive and up-to-date overview of the 3L-CVRP literature is provided by Nascimento, Toledo, and Coelho (2026).

The *Capacitated Profitable Tour Problem* (CPTP) is a variant of vehicle routing in which not every customer has to be visited (see Caspar (2026) for a current review). Instead, the objective becomes to select a subset of customers that maximizes the net profit, defined as the total prize collected minus transportation costs, while respecting vehicle capacity constraints.

Despite the extensive research on 3L-CVRP and profitable routing, these two problem characteristics have not yet been considered simultaneously. In particular, the integration of customer selection decisions from the CPTP with 3D loading constraints introduces additional challenges, as the selection of profitable customers directly affects both routing costs and feasibility. To the best of our knowledge, no previous study has investigated the combination of CPTP and three-dimensional loading constraints in a routing context. While selection and packing decisions are jointly optimized in the knapsack literature, and loading and routing decisions are jointly optimized in the 3L-CVRP literature, the three decisions have not been addressed simultaneously, leaving an important research gap addressed by this work.

2.3. Current Algorithms

The integrated optimization of routing and loading remains computationally challenging, as both the vehicle routing problem and the loading problem are NP-hard, while their joint optimization is considerably more difficult. Consequently, some studies have treated routing and loading as separate problems, either by solving vehicle routing problems without explicitly considering three-dimensional loading constraints or by investigating loading problems independently of routing decisions. Owing to the combinatorial complexity of simultaneously optimizing routing and loading decisions, a wide range of solution approaches has been proposed for the various problem variants that have emerged in the literature.

Building upon the observation of Tamke et al. (2025), up to the year 2023, metaheuristic approaches, particularly tabu search methods, were the most frequently applied techniques for

Table 1 Overview of recent 3L-CVRP solution methods.

Reference	Category	Solution Method
Cen et al. (2023)	Metaheuristic	Adaptive Large Neighborhood Search with Multi-Order Packing Search
Moura et al. (2023)	Matheuristic	Integration of Two Mathematical Models
Chi et al. (2024)	Exact	Branch-and-Price with Dynamic Programming
Tamke et al. (2024)	Exact	Branch-and-Cut with Infeasible Path Elimination
Yin et al. (2025)	Metaheuristic	Genetic Algorithm with Residual Space Optimization
Schoepf et al. (2025)	Learning-based	Proximal Policy Optimization
Nirwan et al. (2025)	Metaheuristic	Genetic Algorithm with Residual Space Optimization
Peric et al. (2025)	Hybrid	Modular Iterative Routing and Packing Algorithm
Vu et al. (2026)	Metaheuristic	Ant Colony Optimization
Nascimento et al. (2026)	Matheuristic	Logic-Based Benders Decomposition
Saavedra et al. (2026)	Metaheuristic	Variable Neighborhood Search

solving 3L-CVRP variants. We review subsequent developments of solutions in the literature, as summarized in Table 1.

Cen et al. (2023) study the 3L-CVRP with cross-docking, in which pickup and delivery routes are jointly optimized while accounting for vehicle capacity, three-dimensional loading, and consolidation constraints. The objective is to minimize the total transportation, service, and cross-docking operation time. To solve the problem, the authors propose a hybrid heuristic that combines an adaptive large neighborhood search with a multi-order local search loading algorithm. Computational experiments are conducted on the benchmark instances of Gendreau et al. (2006) as well as additional benchmark sets.

Moura et al. (2023) investigate the integration of the 3D-BPP with the CVRP featuring simultaneous deliveries and pickups. The objective is to determine a set of routes that minimizes the total distribution and collection costs while satisfying all customer demands, pickup quantities, and operational constraints. The authors formulate two mathematical models that decouple the loading and routing decisions and solve them sequentially. Computational experiments on both real-world and benchmark instances indicate that optimizing packing decisions has a greater impact on overall solution quality than optimizing routing decisions.

Chi, He, and Song (2024) address the 3L-CVRP with relocation constraints, aiming to minimize the total transportation and relocation costs. The authors formulate a MILP model and develop a branch-and-price algorithm to jointly optimize routing and loading decisions while accounting for required relocations. Furthermore, they propose loading algorithms based on dynamic programming and additional enhancement strategies to efficiently handle 3L constraints. Computational experiments on benchmark instances validate the significance of necessary relocations.

Tamke et al. (2025) revisit the 3L-CVRP introduced by Gendreau et al. (2006) by developing a branch-and-cut framework. Their approach proves optimality for many solutions that were previously considered the best-known heuristic results and improves the best-known solutions for several benchmark instances across all five loading variants. However, the scalability of the branch-and-cut algorithm remains limited, with computational challenges arising for instances containing more than 30 customers.

Yin et al. (2025) investigate the 3L-CVRP with multiple trips and without weight constraints, aiming to simultaneously minimize total delivery distance and maximize vehicle loading rates. To solve the problem, the authors propose a genetic algorithm integrated with a packing algorithm based on a residual-space optimization strategy. The proposed approach is evaluated on real-world instances and compared with existing benchmark algorithms.

Schoepf et al. (2025) also revisit the 3L-CVRP introduced by Gendreau et al. (2006) and propose a reinforcement learning approach based on the *Proximal Policy Optimization* (PPO) algorithm. Computational experiments on the corresponding benchmark instances demonstrate the favourable scalability of PPO as the problem size increases, while achieving solution quality comparable to current state-of-the-art methods.

Nirwan and Nirwan (2025) investigate a multi-objective variant of the 3L-CVRP, where the total delivery distance is minimized while vehicle load utilization is maximized through a combined fitness function. The authors propose a hybrid metaheuristic approach that integrates a genetic algorithm with a residual space optimizer to jointly optimize routing decisions and loading feasibility, and evaluate its performance on several benchmark instances.

Similar to Moura et al. (2023), Perić, Kolak, and Lešić (2025) adopt a modular approach for the 3D-BPP-CVRP, aiming to minimize total travel distance by treating routing and 3D-BPP as separate but interrelated subproblems. In particular, Perić, Kolak, and Lešić (2025) propose a modular iterative routing and packing algorithm that achieves substantially lower execution times than more complex state-of-the-art methods on the Gendreau et al. (2006) benchmark instances, while maintaining competitive solution quality.

Vu and Nong (2026) investigate a variant of the 3L-CVRP that minimizes both total travel distance and center-of-mass penalties. They propose a metaheuristic approach that combines minimum spanning tree-based customer clustering, ant colony optimization for route construction, and relocation-based local search. The proposed method is evaluated on the benchmark instances of Gendreau et al. (2006) as well as real-world instances.

Nascimento, Toledo, and Coelho (2026) investigate the 3L-CVRP with time windows, with the objective of minimizing total travel costs and fixed vehicle usage costs. They propose a logic-based Benders decomposition approach that decomposes the problem into a routing master problem and a 3L subproblem. The method iteratively eliminates infeasible routing decisions through feasibility cuts, while incorporating packing heuristics and integer programming techniques to improve computational efficiency. Computational experiments are performed on the benchmark instances of Gendreau et al. (2006) and extended benchmark sets.

Rojas-Saavedra, Álvarez-Martínez, and Escobar (2026) investigate the 3L-CVRP with heterogeneous vehicle fleets and incorporate environmental impact, measured through fuel consumption, into the objective function. They propose a variable neighborhood search metaheuristic enhanced with a greedy randomized adaptive search procedure to efficiently evaluate loading feasibility and solve the integrated routing and loading problem. Computational experiments on benchmark instances show that the proposed approach achieves competitive results, outperforming several previously published methods while requiring relatively short computation times.

Notably, *Simulated Annealing* (SA), a metaheuristic optimization algorithm that iteratively improves solutions by occasionally accepting inferior solutions with a decreasing probability to escape local optima, has demonstrated promising performance in solving various routing problems (see Yu et al. (2022) and references therein), 3D-BPP (see Kacprzak, Rudy, and Żelazny (2015) and references therein), and 3DL problems (see Ma et al. (2025) and references therein). However, to the best of our knowledge, no studies have yet applied SA to the integrated problem of routing and loading.

Building on these observations, we address the identified research gap by introducing the *Three-Dimensional Packing Capacitated Profitable Tour Problem* (3DP-CPTP) that integrates customer selection, vehicle routing, and 3D-BPP constraints into a single optimization problem. Second, we adapt established solution approaches from related problems, namely a matheuristic and a tabu search algorithm, to solve the 3DP-CPTP. Third, we develop a SA algorithm which, to the best of our knowledge, represents the first application of SA to this integrated routing and 3D packing problem. Finally, we conduct an extensive computational study on the well-known benchmark instances of Gendreau et al. (2006) to evaluate the effectiveness and computational efficiency of the proposed solution methods.

3. Problem and Methodology

This section defines the 3DP-CPTP and provides a mathematical MILP formulation.

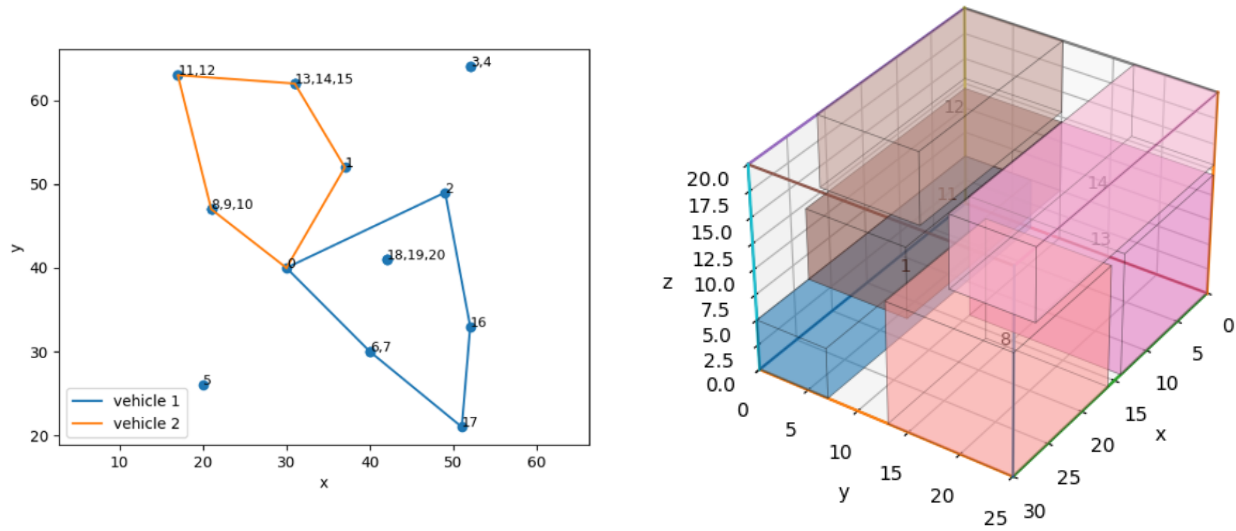


Figure 1 Example for the 3DP-CPTP. The first vehicle transports the items 2, 6, 7, 16 and 17, while the second vehicle transports the items 1, 8, 9, 10, 11, 12, 13, 14 and 15 (items 3, 4, 5, 18, 19 and 20 are rejected). The three-dimensional packing in the second vehicle is visualized in the right part of the figure.

3.1. Problem Definition

The 3DP-CPTP is defined on a complete, weighted graph $\mathcal{G} = (\mathcal{N}_0, \mathcal{A})$, where $\mathcal{N}_0 = \{0, 1, \dots, N\}$ represents a set of vertices, each representing a customer, with 0 denoting the depot, and $\mathcal{N} = \mathcal{N}_0 \setminus \{0\}$ representing $N \in \mathbb{N}$ potential customers. The set of arcs is given by $\mathcal{A} = \{(u, v) : u \neq v, u, v \in \mathcal{N}_0\}$, while each arc is associated with a symmetric, non-negative travel cost $c_{u,v} = c_{v,u} \in \mathbb{R}_{\geq 0}$.

A fleet of vehicles $\mathcal{K} = \{1, \dots, K\}$ is stationed at the depot. Each vehicle $k \in \mathcal{K}$ has a maximum weight capacity $Q_k \in \mathbb{R}_{\geq 0}$ and a finite three-dimensional rectangular space with dimensions $(D_k^x, D_k^y, D_k^z) \in \mathbb{R}_{\geq 0}^3$.

Each customer $u \in \mathcal{N}$ has one or more rectangular items, while each item $i \in \mathcal{I} = \{1, \dots, I\}$ is associated with a prize p_i , which is gained if the item is packed and transported in a vehicle. Each rectangular item i can rotate according to the six orthogonal orientations $\mathcal{R} = \{1, \dots, 6\}$ obtained by permuting its length l_i , width w_i , and height h_i leading to a space usage of $(d_{i,r}^x, d_{i,r}^y, d_{i,r}^z)$.

Consequently, the objective of the 3DP-CPTP is to maximize the net profit, defined as the total prize collected from the selected items minus the total routing costs. To achieve this objective, the 3DP-CPTP requires determining a feasible solution that simultaneously specifies:

1. the subset of accepted items,
2. the assignment of accepted items to vehicles,
3. the tour of each used vehicle, and

4. a feasible packing of the assigned items inside the corresponding vehicle, while the packing must satisfy non-overlap, container-boundary, orientation, and capacity weight restrictions.

The parameters and decision variables used in the mathematical formulation of the 3DP-CPTP are summarized in Table 2.

Table 2 Definitions of parameters and variables used in the 3DP-CPTP formulation.

Parameters:	
$\mathcal{N} = \{1, \dots, N\}$	Customers
$\mathcal{N}_0 = \mathcal{N} \cup \{0\}$	Customers and depot
$\mathcal{K} = \{1, \dots, K\}$	Vehicles
$\mathcal{A} = \{(u, v) : u \neq v, u, v \in \mathcal{N}_0\}$	Arcs between customers and depot
$\mathcal{I} = \{1, \dots, I\}$	Items
$\mathcal{R} = \{1, \dots, 6\}$	Item orientation types
$\mathcal{S} \subseteq \mathcal{N}$	Subsets of customers ($ \mathcal{S} \geq 2$)
$\mathcal{D} = \{x, y, z\}$	Dimensions
$c_{u,v}$	Cost of traveling on arc (u, v)
p_i	Prize from accepting item i
(l_i, w_i, h_i)	Length, width, height of item i
$(d_{i,r}^x, d_{i,r}^y, d_{i,r}^z)$	Space usage of item i for orientation $r \in \mathcal{R}$
q_i	Weight quantity of item i
(D_k^x, D_k^y, D_k^z)	Container space dimensions of vehicle k
Q_k	Weight capacity of vehicle k
(M^x, M^y, M^z)	Container space dimension specific big-M values
Variables:	
$x_{u,v,k} \in \{0, 1\}$	Equals 1 iff arc (u, v) is part of the tour of vehicle k
$y_{u,k} \in \{0, 1\}$	Equals 1 iff vehicle k travels to customer u
$z_i \in \{0, 1\}$	Equals 1 iff item i is accepted
$o_{i,k}^r \in \{0, 1\}$	Equals 1 iff item i is oriented to type r in vehicle k
$a_{i,j,k}^\alpha \in \{0, 1\}$	Equals 1 iff item i is placed before item j along axis α
$v_k \in \{0, 1\}$	Equals 1 iff vehicle k is used
$b_{i,k}^\alpha \geq 0$	Space usage of item i in dimension α in vehicle k
$\lambda_{i,k} \in \{0, 1\}$	Equals 1 iff item i is loaded into vehicle k

3.2. Mixed Integer Linear Programming Formulation

Given the function $\pi : \mathcal{I} \rightarrow \mathcal{N}$, which associates each item with a specific customer, the MILP model for the 3DP-CPTP (1)–(24) becomes:

$$\max \sum_{i \in \mathcal{I}} p_i z_i - \sum_{k \in \mathcal{K}} \sum_{(u,v) \in \mathcal{A}} c_{u,v} x_{u,v,k} \quad (1)$$

$$\sum_{k \in \mathcal{K}} \lambda_{i,k} = z_i \quad \forall i \in \mathcal{I} \quad (2)$$

$$\lambda_{i,k} \leq y_{\pi(i),k} \quad \forall i \in \mathcal{I}, k \in \mathcal{K} \quad (3)$$

$$y_{u,k} \leq \sum_{i \in \mathcal{I} : \pi(i)=u} \lambda_{i,k} \quad \forall u \in \mathcal{N}, k \in \mathcal{K} \quad (4)$$

$$\sum_{v \in \mathcal{N}_0 \setminus \{u\}} x_{u,v,k} = y_{u,k} \quad \forall u \in \mathcal{N}, k \in \mathcal{K} \quad (5a)$$

$$\sum_{v \in \mathcal{N}_0 \setminus \{u\}} x_{v,u,k} = y_{u,k} \quad \forall u \in \mathcal{N}, k \in \mathcal{K} \quad (5b)$$

$$\sum_{v \in \mathcal{N}} x_{0,v,k} = v_k \quad \forall k \in \mathcal{K} \quad (6a)$$

$$\sum_{u \in \mathcal{N}} x_{u,0,k} = v_k \quad \forall k \in \mathcal{K} \quad (6b)$$

$$y_{u,k} \leq v_k \quad \forall u \in \mathcal{N}, k \in \mathcal{K} \quad (7)$$

$$\sum_{u \in S} \sum_{v \in S \setminus \{u\}} x_{u,v,k} \leq |S| - 1 \quad \forall S \subseteq \mathcal{N}, k \in \mathcal{K} \quad (8)$$

$$\sum_{i \in \mathcal{I}} q_i \lambda_{i,k} \leq Q_k \quad \forall k \in \mathcal{K} \quad (9)$$

$$\sum_{r \in \mathcal{R}} o_{i,k}^r = \lambda_{i,k} \quad \forall i \in \mathcal{I}, k \in \mathcal{K} \quad (10)$$

$$b_{i,k}^\alpha + \sum_{r \in \mathcal{R}} d_{i,r}^\alpha o_{i,k}^r \leq D_k^\alpha \quad \forall i \in \mathcal{I}, k \in \mathcal{K}, \alpha \in \mathcal{D} \quad (11)$$

$$b_{i,k}^\alpha \leq D_k^\alpha \lambda_{i,k} \quad \forall \alpha \in \mathcal{D}, i \in \mathcal{I}, k \in \mathcal{K} \quad (12)$$

$$a_{i,j,k}^\alpha \leq \lambda_{i,k} \quad \forall \alpha \in \mathcal{D}, k \in \mathcal{K}, i, j \in \mathcal{I} (i \neq j) \quad (13a)$$

$$a_{i,j,k}^\alpha \leq \lambda_{j,k} \quad \forall \alpha \in \mathcal{D}, k \in \mathcal{K}, i, j \in \mathcal{I} (i \neq j) \quad (13b)$$

$$a_{i,j,k}^\alpha + a_{j,i,k}^\alpha \leq 1 \quad \forall \alpha \in \mathcal{D}, i, j \in \mathcal{I} (i < j), k \in \mathcal{K} \quad (14)$$

$$\sum_{\alpha \in \mathcal{D}} (a_{i,j,k}^\alpha + a_{j,i,k}^\alpha) \geq \lambda_{i,k} + \lambda_{j,k} - 1 \quad \forall i, j \in \mathcal{I} (i < j), k \in \mathcal{K} \quad (15)$$

$$b_{i,k}^\alpha + \sum_{r \in \mathcal{R}} d_{i,r}^\alpha o_{i,k}^r \leq b_{j,k}^\alpha + M^\alpha (1 - a_{i,j,k}^\alpha) \quad \forall \alpha \in \mathcal{D}, k \in \mathcal{K}, i, j \in \mathcal{I} (i \neq j) \quad (16)$$

$$v_k \in \{0, 1\} \quad \forall k \in \mathcal{K} \quad (17)$$

$$y_{u,k} \in \{0, 1\} \quad \forall u \in \mathcal{N}, k \in \mathcal{K} \quad (18)$$

$$z_i \in \{0, 1\} \quad \forall i \in \mathcal{I} \quad (19)$$

$$\lambda_{i,k} \in \{0, 1\} \quad \forall i \in \mathcal{I}, k \in \mathcal{K} \quad (20)$$

$$x_{u,v,k} \in \{0, 1\} \quad \forall k \in \mathcal{K}, u, v \in \mathcal{N}_0, u \neq v \quad (21)$$

$$o_{i,k}^r \in \{0, 1\} \quad \forall i \in \mathcal{I}, k \in \mathcal{K}, r \in \mathcal{R} \quad (22)$$

$$a_{i,j,k}^\alpha \in \{0, 1\} \quad \forall \alpha \in \mathcal{D}, i, j \in \mathcal{I}, i \neq j, k \in \mathcal{K} \quad (23)$$

$$b_{i,k}^\alpha \geq 0 \quad \forall \alpha \in \mathcal{D}, i \in \mathcal{I}, k \in \mathcal{K} \quad (24)$$

The objective function (1) maximizes the difference between the gained prize over all accepted items and the transportation costs. Equations (2) ensure that each accepted item is loaded into exactly one vehicle, while rejected items are assigned to none. Constraints (3) link the packing and routing decisions by allowing item i to be loaded onto vehicle k only if vehicle k visits the corresponding customer $\pi(i)$. Inequalities (4) prevent empty customer visits by requiring vehicle k to carry at

least one item associated with customer u whenever it visits u . Together, these constraints decouple item assignment from customer assignment, allowing a customer's items to be distributed among several vehicles, with each vehicle carrying a subset of the corresponding items. Equations (5a) and (5b) ensure that for each customer u and vehicle k , whenever customer u is served by vehicle k , exactly one incoming arc and one outgoing arc are assigned to u . The vehicle activation equations (6a) and (6b) guarantee that iff vehicle k leaves and visits the depot, it must be part of the solution ($v_k = 1$). Inequalities (7) link customer visits to vehicle usage to strengthen the linear relaxation. For every vehicle k and every subset of customers u , the subtour elimination constraints (8) guarantee connectivity to the depot. Inequalities (9) state that the total weight of all accepted items cannot exceed the weight capacity of a vehicle. The equations (10) guarantee that if vehicle k accepts item i , exactly one of the six possible item orientations $o_{i,k}^r$ is selected. Constraints (11) ensure that every packed item remains entirely within the bounds of its assigned vehicle container space in all dimensions. Inequalities (12) bound the position of each item by the corresponding container dimension and fix the position variables of items not loaded into vehicle k to zero. It removes degenerate solutions in which unloaded items are assigned arbitrary positions and thereby strengthens the linear relaxation. Inequalities (13a) and (13b) ensure that precedence variables are set to zero unless the corresponding items are assigned to the same vehicle. Restrictions (14) guarantee that items cannot both be placed before each other along the same dimension. Inequalities (15) ensure that if both items i and j are packed in vehicle k ($\lambda_{i,k} = \lambda_{j,k} = 1$), then at least one of the six corresponding a variables must equal one, implying that the two items are separated along at least one dimension. Inequalities (16) enforce non-overlapping conditions between items, as if $a_{i,j,k}^\alpha = 1$, then item i must be positioned before item j along dimension α . Finally, the domains of the variables are defined by restrictions (17)–(24).

3.3. Matheuristic

Matheuristics are solution methods that embed mathematical programming inside a heuristic framework and have been applied to many different routing problems (Schermer, Moeini, and Wendt (2019)). The main notation used in our *Matheuristic* (MH) framework is summarized in Table 3.

A destroy strategy in the proposed matheuristic partially relaxes the incumbent solution, allowing the repair phase to reoptimize a subset of decision variables. It defines the optimization neighborhood by specifying which variables remain fixed at their incumbent values and which are released for optimization. We consider four destroy strategies, each constructing a primary destroy set $D \subseteq S^1$ by selecting accepted items according to a different criterion.

Table 3 Common notation for the algorithm Matheuristic

Symbol	Explanation
I^{max}	Maximum number of iterations
T_0	Time limit for the initial MILP solve
T_t	Time limit for each iteration
ρ	Destroy fraction, to free currently accepted items
δ	Distance threshold defining the destroy neighborhood
σ_{MH}^{max}	Stagnation threshold (non-improving iterations before increasing the destroy fraction)
S	Incumbent solution
S^1	Set of accepted items in the incumbent solution S
S^0	Set of rejected items in the incumbent solution S
D	Destroy set, subset of items whose decisions are released in the current iteration
E	Set of rejected items temporarily released for reoptimization and not part of the destroy set
d	Size of the destroy set (number of released items)
γ	Destroy-fraction contraction factor used after an improving iteration (default: 0.8)
η	Destroy-fraction expansion increment used after stagnation (default: 0.1)

1. **Random:** Uniformly samples d items from S^1 . If $S^1 = \emptyset$, sample items randomly to promote diversification.

2. **Worst-Profit:** Computes an item contribution score:

$$\pi_i \approx p_i - \sum_{k \in K} \sum_{j \in I \setminus \{i\}} \left(c_{\pi(i), \pi(j)} x_{\pi(i), \pi(j), k} + c_{\pi(j), \pi(i)} x_{\pi(j), \pi(i), k} \right). \quad (2)$$

for each accepted item. The subsequent repair phase may replace these items with more profitable alternatives or reconstruct more efficient routes.

3. **Spatial-Cluster:** Selects one accepted item at random and iteratively removes its nearest neighbors until d items have been removed. This strategy exploits the spatial structure of capacitated profitable routing, where geographically close items are likely to belong to the same routes and vehicle assignments.

4. **Route-Based:** Removes all items from selected routes until at least d items have been removed. By relaxing entire routes, this strategy enables the repair phase to reassign items across vehicles and reconstruct tours, helping the search escape poor route decompositions.

The following Figure 2 visualizes these four destroy strategies that are used in the matheuristic.

The matheuristic to provide a solution for the 3DP-CPTP is given with Algorithm 1.

Algorithm 1 first computes a feasible incumbent solution of the full model (1)–(24) within the time limit T_0 . At each subsequent iteration, a destroy set D of items is constructed using one of the strategies shown in Figure 2, enabling the search to explore different regions of the solution space. In addition, the set E contains nearby currently unaccepted items, enabling the restricted MILP not

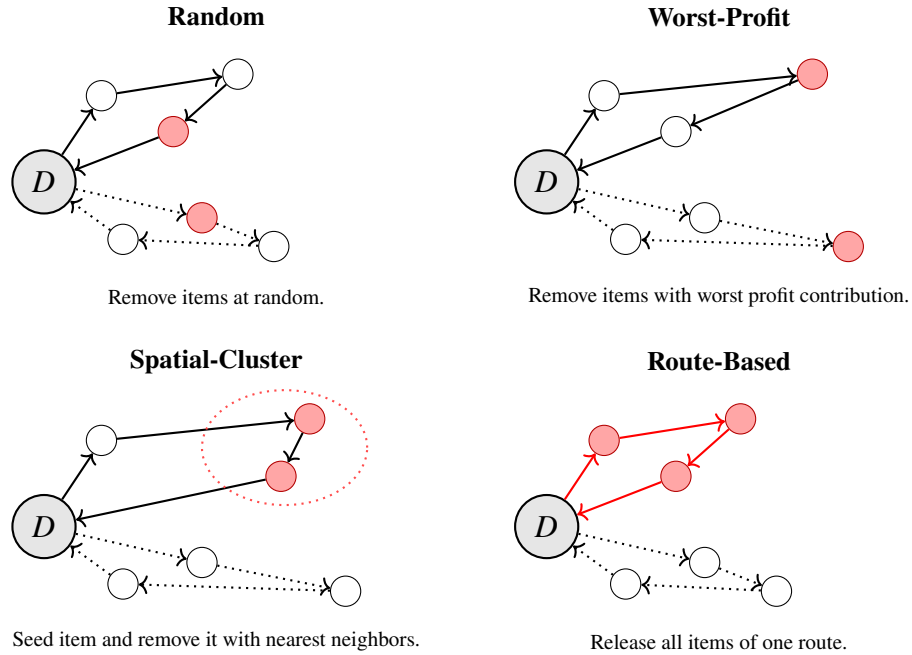


Figure 2 Illustration of the four destroy operators used in the matheuristic (see Algorithm 1).

only to reconfigure the served items but also to insert promising unserved ones (see line 11). Based on the sets D and E , routing and packing decisions associated with these items are released, while all remaining decisions are fixed to their incumbent values, resulting in a smaller restricted MILP that is solved at each iteration (lines 18-26). If an improved solution is found (see line 28), the incumbent solution is updated. Otherwise, after σ_{MH}^{max} non-improving iterations, the destroy size d is increased to enlarge the search neighborhood and encourage greater exploration in subsequent iterations.

3.4. Metaheuristics

While the MILP and matheuristic operate at the item level, metaheuristics can also take the customer as the atomic unit of every neighborhood move, as all items requested by a customer are assigned to the same vehicle, or the customer is not served. Accordingly, a metaheuristic solution is represented as a set of routes, where each route is an ordered sequence of customers, with all items of each customer packed onto one corresponding vehicle. Customer-based decisions are imposed for three reasons. First, splitting a customer's items across multiple vehicles requires repeated visits to the same delivery location and entails duplicated service effort that is often not beneficial. Second, customer-level decisions substantially reduce the search space. When each customer requests multiple items, treating the customer as an indivisible unit replaces multiple item-level assignment decisions with a single customer-level decision, thereby substantially reducing

Algorithm 1 Matheuristic for the 3DP-CPTP

```

1: procedure MATHEURISTIC( $I^{max}, \rho_0, \rho^{max}, T_0, T_t, \delta, \sigma_{MH}^{max}, \gamma, \eta$ )
2:   Solve MILP for time  $T_0$  to obtain incumbent solution  $S$ 
3:    $S^* \leftarrow S, \quad f^* \leftarrow f(S), \quad \rho \leftarrow \rho_0, \quad \sigma \leftarrow 0$ 
4:   for  $t \leftarrow 1$  to  $I^{max}$  do
5:     Extract  $S^1, S^0, x, y, z, v$  from  $S^*$ 
6:      $d \leftarrow \max\{2, \lfloor \rho \cdot \max(|S^1|, 1) \rfloor\}$ 
7:     Select destroy strategy to build destroy set  $D$  ▷ see Figure 2
8:      $E \leftarrow \emptyset$ 
9:     for each item  $i \in D$  do
10:      for each item  $j \in S^0$  and  $j \notin D$  and  $dist(i, j) < \delta$  do
11:        add item  $j$  to  $E$  ▷ Add nearby currently unserved items
12:      end for
13:    end for
14:    if  $|E| > d$  then
15:      reduce  $E$  to a random subset of size  $d$ 
16:    end if
17:     $D \leftarrow D \cup E$ 
18:    for each item  $i \in N$  do ▷ Free and fix variables
19:      free  $z_i, y_{i,k}, p_{i,k}^x, p_{i,k}^y, p_{i,k}^z, o_{i,k,r}$  if  $i \in D$ , otherwise fix them
20:    end for
21:    for each arc  $(i, j, k)$  do
22:      fix  $x_{i,j,k}$  if  $i, j \notin D$  and  $i, j \neq 0$ , otherwise free it
23:    end for
24:    for each vehicle  $k$  do
25:      free  $v_k$  if  $k$  is affected by  $D$  or currently unused; otherwise fix it
26:    end for
27:    Load  $S^*$  and solve restricted MILP for time  $T_t$  ▷ Solve current iteration
28:    if feasible  $S'$  found and  $f(S') > f^*$  then
29:       $S^* \leftarrow S', \quad f^* \leftarrow f(S'), \quad \sigma \leftarrow 0, \quad \rho \leftarrow \max\{\rho_0, \gamma\rho\}$ 
30:    else
31:       $\sigma \leftarrow \sigma + 1$ 
32:    end if
33:    if  $\sigma \geq \sigma_{MH}^{max}$  then ▷ Too many non-improving iterations
34:       $\rho \leftarrow \min\{\rho^{max}, \rho + \eta\}, \quad \sigma \leftarrow 0$ 
35:    end if
36:  end for
37:  return  $(S^*, f^*)$ 
38: end procedure

```

the search space. Consequently, for a fixed computational budget, the metaheuristic can explore a larger fraction of the relevant search space, an advantage that becomes increasingly important for larger instances. Third, customer-level moves improve neighborhood connectivity. Under an item-level move set, transferring a customer between vehicles may require several consecutive item moves, with intermediate solutions in which the customer's items are split across vehicles. These intermediate solutions may incur duplicated visits and substantially higher routing costs than either endpoint, making them unlikely to be accepted and effectively obstructing the transition. By treating the customer as the atomic unit, the same transformation can be performed in a single move, thereby directly connecting structurally distinct high-quality solutions in the search space. Computational experiments in the following section investigate whether customer- or item-based decisions in metaheuristics are more effective for solving the 3DP-CPTP.

Tabu Search (TS) is a metaheuristic that guides the search process by temporarily classifying selected moves or solution attributes as tabu, thereby preventing cycling and encouraging exploration of new regions of the solution space (see Glover (1986)). The main notation used in the TS framework is summarized in Table 4.

Table 4 Common notation for tabu search

Symbol	Explanation
T^{max}	Time limit in seconds
T_{TS}^c	Time limit for checking 3D packing feasibility in seconds for TS
$\mathcal{T}$	Tabu list
$a(m)$	Tabu attribute associated with move m , stored in the tabu list $\mathcal{T}$
$\Delta f(m)$	Predicted change in objective value produced by move m
θ	Tabu tenure (number of iterations a tabu attribute remains tabu)
$\mathcal{N}(S)$	Neighborhood of the current solution S
$f(S)$	Objective value of solution S
σ_{TS}^{max}	Diversification threshold (non-improving iterations before diversification)
R^n	Random subset of accepted items that can be removed during diversification.
n^r	Number of accepted items removed in the diversification phase (default 30%).
U	List of currently rejected items considered for reinsertion.
pos_{CI}	Best insertion index of an item according to the method CHEAPEST-INSERTION().
Δ_{CI}	Incremental routing cost induced by the method CHEAPEST-INSERTION().

In TS, the neighborhood is generated by applying predefined move operators to the current solution. Building on the approach of Gendreau et al. (2006) and insights obtained from extensive computational experiments, five different move operators are considered.

1. **Swap**: For each accepted item and each nearest-neighbor item assigned to a different route, evaluates a swap by checking capacity feasibility and estimating the travel-cost variation obtained by removing and reinserting it at its cheapest feasible position in other routes (see Figure 3).

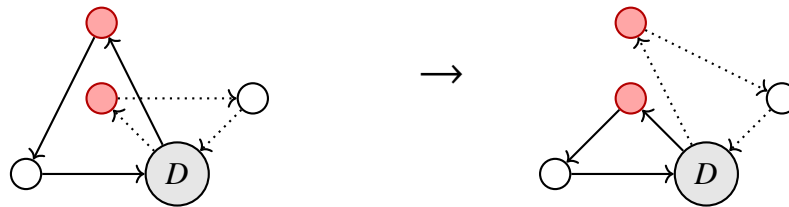


Figure 3 Swap operator.

2. **2-opt**: Selects a route with at least three items and reverses a subsequence to reduce travel cost by eliminating internal route crossings (see Figure 4).

3. **Relocate**: Moves an accepted item from its current route to another route containing at least one of its nearest neighbors (see Figure 5).

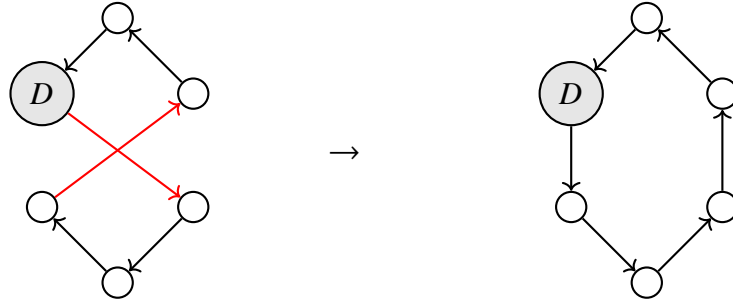


Figure 4 2-opt operator.

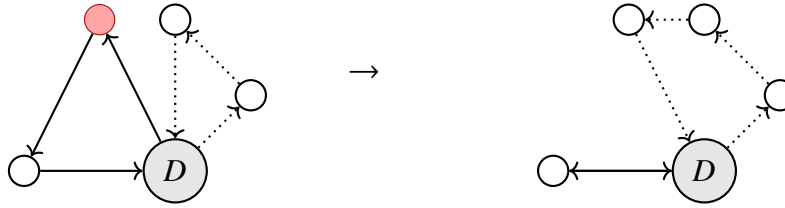


Figure 5 Relocate operator.

4. **Insert:** Attempts to insert a rejected item into routes containing one of its nearest neighbors, or into any route if none of these neighbors is currently served (see Figure 6).

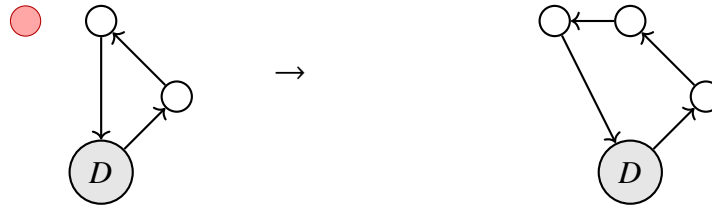


Figure 6 Insert operator.

5. **Remove:** Remove an accepted item from its route as shown in Figure 7.

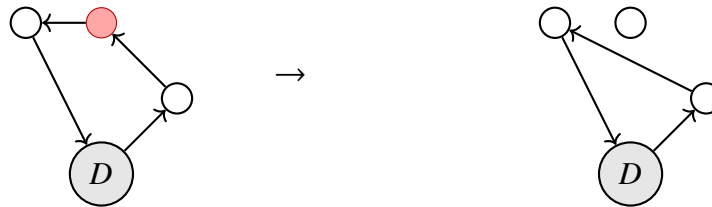


Figure 7 Remove operator.

Algorithm 2 presents the proposed tabu search approach for solving the 3DP-CPTP.

The tabu search algorithm (see Algorithm 2) initializes a tabu list $\mathcal{T}$ that stores move attributes for a fixed tenure to prevent immediate undoing. First, a 3DP-CPTP solution is constructed using a greedy heuristic inspired by Clarke and Wright (1964), which ranks items according to the ratio of

Algorithm 2 Tabu search for the 3DP-CPTP

```

1: procedure TABUSEARCH( $T^{\max}, T_{TS}^c, \theta, \sigma_{TS}^{\max}$ )
2:   Initialize tabu list  $\mathcal{T}$ 
3:   Construct initial solution  $S$  via greedy construction heuristic ▷ see Clarke and Wright (1964)
4:    $S^* \leftarrow S, \quad f^* \leftarrow f(S), \quad \sigma \leftarrow 0, \quad t \leftarrow 1, \quad T \leftarrow 0$ 
5:   while elapsed time  $\leq T^{\max}$  do
6:     Generate neighborhood  $\mathcal{N}(S)$  via the move operators ▷ see Figure 3–7
7:     if  $\mathcal{N}(S)$  is empty then
8:       break
9:     end if
10:    Sort  $\mathcal{N}(S)$  by decreasing  $\Delta f(m)$ 
11:     $m^* \leftarrow \emptyset$  ▷ Moves for the current iteration
12:    for all  $m \in \mathcal{N}(S)$  do
13:      Compute tabu attribute  $a(m)$ 
14:      if  $a(m) \in \mathcal{T}$  and  $f(S) + \Delta f(m) \leq f^*$  then
15:        continue ▷ Move  $m$  is tabu and not increasing solution  $S$ 
16:      end if
17:      if applying  $m$  to  $S$  is feasible in time limit  $T_{TS}^c$  then
18:         $m^* \leftarrow m$  ▷ Check 3D packing-feasible
19:      end if
20:    end for
21:    if  $m^* = \emptyset$  then ▷ No feasible solution according to moves
22:       $\sigma \leftarrow \sigma + 1$ 
23:    else
24:      Apply  $m^*$  to obtain new  $S$  and recompute  $f(S)$ 
25:      Insert  $m^*$  into  $\mathcal{T}$  with expiry  $t + \theta$ 
26:      if  $f(S) > f^*$  then ▷ Better solution is found
27:         $S^* \leftarrow S, \quad f^* \leftarrow f(S), \quad \sigma \leftarrow 0$ 
28:      else
29:         $\sigma \leftarrow \sigma + 1$ 
30:      end if
31:    end if
32:    if  $\sigma \geq \sigma_{TS}^{\max}$  then
33:       $S \leftarrow \text{DIVERSIFY}(S)$  ▷ see Algorithm 4
34:      Recompute  $f(S)$ 
35:       $\sigma \leftarrow 0$ 
36:    end if
37:    Remove expired entries from  $\mathcal{T}$  ▷ Update tabu list
38:     $t \leftarrow t + 1$ 
39:  end while
40:  return ( $S^*, f^*$ )
41: end procedure

```

their prize to the associated transportation cost and then greedily inserts them into feasible routes. At each iteration (see line 5), a neighborhood of the current solution S is generated using the five move operators (see Figure 3–7). Each move is ranked according to its predicted change in objective value (1) (see line 10). If a move m is tabu and does not improve the current best solution, it is discarded (see line 15). Otherwise, if the move is feasible (i.e., a feasible 3D packing exists for all items), it is added to the current move set m^* . All selected moves are applied to obtain a new solution, which is compared against the best solution found so far and updated if improved. If no improvement is observed for $\sigma_{TS}^{\max}$ consecutive iterations, a diversification procedure is triggered (line 33), where accepted items are removed and subsequently reinserted to explore new regions of

the search space (see Algorithm 4 in Appendix A for details). Finally, expired entries are removed from the tabu list at the end of each iteration.

SA is a metaheuristic, proposed by Kirkpatrick, Gelatt Jr, and Vecchi (1983), based on the analogy between the annealing process of solids and the problem of solving combinatorial optimization. SA iteratively improves solutions by occasionally accepting inferior solutions with a decreasing probability to escape local optima. The main notation used in the SA framework is summarized in Table 5.

Table 5 Common notation for simulated annealing

Symbol	Explanation
T_{SA}^c	Time limit for checking 3D packing feasibility in seconds for SA
T^{max}	Time limit in seconds
$Temp_t$	Current temperature in the simulated annealing process at iteration t
σ_{SA}^{max}	Diversification threshold (non-improving iterations before diversification and reheating)
α	Cooling factor to update temperature $Temp_t$
β	Reheating factor to update temperature $Temp_t$

Following the five different move operators (see Figure 3–7) and the diversify function (see Algorithm 4 in Appendix A for details) of the TS method from the previous subsection, we describe our SA method in Algorithm 3.

The simulated annealing algorithm (see Algorithm 3) also initializes a feasible solution by the greedy heuristic inspired by Clarke and Wright (1964). At each iteration (see line 4), a neighborhood of the current solution S is generated using the five move operators (see Figure 3-7). Compared to tabu search (see Algorithm 2), simulated annealing ranks candidate moves in a random order, enabling a stochastic exploration of the search space. A move m is accepted whenever it yields a positive change in the objective value. Otherwise, it is accepted with probability $\exp(\Delta f(m)/Temp_t)$ (line 16), allowing occasional acceptance of non-improving moves to escape local optima. If an accepted move is also feasible (i.e., a feasible 3D packing exists), it is added to the current move set m^* . After evaluating all candidate moves, the selected moves are applied to obtain a new solution that is compared with the best solution found so far, and updated if an improvement is achieved. Subsequently, the current temperature $Temp_t$ is reduced according to the cooling factor α for the next iteration. If no improvement is observed for σ_{SA}^{max} consecutive iterations, reheating is performed (see line 34), and a diversification procedure is triggered (line 33), where accepted items are removed and subsequently reinserted to explore new regions of the search space (see Algorithm 4 in Appendix A for details).

Algorithm 3 Simulated annealing for the 3DP-CPTP

```

1: procedure SIMULATEDANNEALING( $T^{\max}, T_{SA}^c, \text{Temp}_0, \alpha, \sigma_{SA}^{\max}, \beta$ )
2:   Construct an initial solution  $S$  via greedy construction heuristic ▷ see Clarke and Wright (1964)
3:    $S^* \leftarrow S, f^* \leftarrow f(S), f \leftarrow f(S), t \leftarrow 1, \text{Temp}_t \leftarrow \text{Temp}_0, \sigma \leftarrow 0, T \leftarrow 0$ 
4:   while elapsed time  $\leq T^{\max}$  do
5:     Draw a single move  $m$  from  $\mathcal{N}(S)$ 
6:     if  $m = \emptyset$  then ▷ No operator yields a move
7:        $\sigma \leftarrow \sigma + 1$ 
8:     else
9:       Compute objective variation  $\Delta f(m)$ 
10:      if  $\Delta f(m) > 0$  then
11:        Accept  $m$  ▷ Improving move
12:      else
13:        Accept  $m$  with probability  $\exp(\Delta f(m)/\text{Temp}_t)$ 
14:      end if
15:      if  $m$  is accepted then
16:         $S' \leftarrow S$  with  $m$  applied
17:        if all routes affected by  $m$  are packing-feasible in time limit  $T_{SA}^c$  then
18:           $S \leftarrow S', f \leftarrow f + \Delta f(m)$  ▷ Incremental objective update
19:          if  $f > f^*$  then ▷ Better solution is found
20:             $S^* \leftarrow S, f^* \leftarrow f, \sigma \leftarrow 0$ 
21:          else
22:             $\sigma \leftarrow \sigma + 1$ 
23:          end if
24:        else
25:           $\sigma \leftarrow \sigma + 1$  ▷ Move rejected,  $S$  unchanged
26:        end if
27:      else
28:         $\sigma \leftarrow \sigma + 1$  ▷ Move rejected,  $S$  unchanged
29:      end if
30:    end if
31:     $\text{Temp}_t \leftarrow \max(\alpha \text{Temp}_t, \text{Temp}^{\min})$  ▷ Cooling in every iteration
32:    if  $\sigma \geq \sigma_{SA}^{\max}$  then
33:       $\text{Temp}_t \leftarrow \beta \text{Temp}_0$  ▷ Reheating
34:       $S \leftarrow \text{DIVERSIFY}(S)$  ▷ see Algorithm 4
35:       $f \leftarrow f(S)$ 
36:       $\sigma \leftarrow 0$ 
37:    end if
38:     $t \leftarrow t + 1$ 
39:  end while
40:  return  $(S^*, f^*)$ 
41: end procedure

```

4. Computational Experiments

In this section, we first describe the parameter settings used in the experiments and then present the results of the computational experiments.

Computational experiments are based on the instances introduced by Gendreau et al. (2006), specifically E016-03m–E101-14s. The depot, customer locations, item characteristics, and vehicles are kept identical to those defined in the original instances. Regarding the prizes p_i , following Angelelli et al. (2021) and Caspar (2026), we consider four different cases that lead to 108 scenarios:

- P1: All prizes are equal to 1.
- P2: Pseudo-random ($p_i = 1 + ((7141 \cdot i + 73) \bmod 100) \forall i \in \mathcal{V}_N$) (Fischetti, González, and Toth 1998).

- P3: Distance-based ($p_i = 1 + \left\lceil \frac{99 \cdot c_{0,i}}{\max_{j \in \mathcal{V}_N} c_{0,j}} \right\rceil$).
- P4: Proportional to the depot distance ($p_i = c_{0,i} \forall i \in \mathcal{V}_N$).

We use the HPC cluster Elwetritsch of the RPTU Kaiserslautern-Landau, which consists of several AMD EPYC 9135 processors (Note that a single thread of the cluster is comparable to that of a standard desktop computer processor). For each experiment, 1800 CPU seconds are available.

4.1. Algorithmic Parameter Setting

The performance of matheuristics and metaheuristics, such as TS and SA, is highly sensitive to the choice of algorithmic parameters, as demonstrated in previous studies (see Cabrera-Guerrero et al. (2017), Xu, Chiu, and Glover (1998), Jackson, Özcan, and John (2017)). Parameters such as the cooling factor, tabu tenure, and neighborhood size strongly influence the search process and, consequently, the solution quality. Therefore, a comprehensive parameter calibration over a diverse set of instances is essential to identify robust parameter settings. For SA, we do not calibrate the stagnation threshold independently, but couple it to the cooling factor as $\sigma_{SA}^{max} = \ln(T_{min}/T_0)/\ln(\alpha)$ to denote the length of one cooling sweep, i.e., the number of iterations required for the temperature to fall from T_0 (default: 100) to T_{min} (default: 0.1).

Based on intensive experiments (see Section 4.3), the following parameters lead to the best average results according to our computational experiments on 108 scenarios:

Table 6 Best parameter setting for the matheuristic (see Algorithm 1), tabu search (see Algorithm 2), and simulated annealing (see Algorithm 3).

Matheuristic			Tabu search			Simulated annealing		
I^{max}	T^{max}	σ_{MH}^{max}	θ	T_{TS}^c	σ_{TS}^{max}	α	T_{SA}^c	β
30	50	5	10	0.1	300	0.9999	0.05	0.1

4.2. Experimental Results

In Table 7, we present the results obtained by testing our MILP formulation (1)–(24), matheuristic (see Algorithm 1), and tabu search (see Algorithm 2) on 108 scenarios.

The first five entries in each row of Table 7 give the name of the original Gendreau et al. (2006) instance, the number of customers N , the total number of items I , the number of vehicles K , and the prize scenario P . The next four columns state the objective value Z_{MILP} of the MILP formulation and Z_{MH} of the matheuristic as their corresponding optimality gaps (in %) after 1800 CPU seconds. Columns ten and eleven show the objective value of the customer-based tabu search Z_{TS}^c and simulated annealing Z_{SA}^c .

Table 7: Comparative analysis of the MILP model (1)–(24), Matheuristic (see Algorithm 1), customer-based TS (see Algorithm 2) and customer-based SA (see Algorithm 3) with the parameter configurations from Table 6 and a computation time of 1800 CPU seconds.

Instance	N	I	K	P	MILP		Matheuristic		Metaheuristics	
					Z_{MILP}	gap (%)	Z_{MH}	gap (%)	Z_{TS}^c	Z_{SA}^c
E016-03m	15	32	5	1	2658.22	8.2	2800.45	4.7	2767.91	2764.61
E016-03m	15	32	5	2	1233.15	13.8	1286.83	9.3	1242.68	1244.96
E016-03m	15	32	5	3	1544.78	9.2	1577.36	10.1	1572.65	1572.65
E016-03m	15	32	5	4	302.75	21.9	240.08	64.9	303.36	303.36
E016-05m	15	26	5	1	2048.2	4.7	2051.18	4.3	2065.52	2065.52
E016-05m	15	26	5	2	998.49	3	984.57	4.5	953.72	953.72
E016-05m	15	26	5	3	1241.87	2.5	1236.53	3.5	1213.9	1213.9
E016-05m	15	26	5	4	205.13	8.3	205.13	9.5	184.6	184.6
E021-04m	20	37	5	1	3044.75	12.7	3236.6	8.3	3335.8	3337.73
E021-04m	20	37	5	2	1401.37	15.3	1493.46	12	1499.27	1499.27
E021-04m	20	37	5	3	2259.32	16.5	2454.43	8.3	2487.55	2489.73
E021-04m	20	37	5	4	529.34	32.9	530.2	36.6	559.92	559.92
E021-06m	20	36	6	1	2869.22	15.1	3132.18	6.1	3148.5	3159.32
E021-06m	20	36	6	2	1311.58	14	1350.89	11.4	1351.85	1360.93
E021-06m	20	36	6	3	1969.21	21.4	2221.43	8.2	2245.03	2246.32
E021-06m	20	36	6	4	328.65	76.4	441.06	33.9	452.88	452.88
E022-04g	21	45	7	1	1147.03	275.5	4066.22	7.8	4104.3	4104.36
E022-04g	21	45	7	2	1297.56	60.1	1802.84	19.1	1871.41	1869.36
E022-04g	21	45	7	3	431.49	438.5	2103.54	14	2121.3	2121.36
E022-04g	21	45	7	4	70.97	1316.5	776.52	42.1	825.81	831.05
E022-06m	21	40	6	1	3187.27	17.2	3463.43	8.5	3501.56	3504.15
E022-06m	21	40	6	2	1263.93	35.9	1442.17	22.8	1518.16	1518.3
E022-06m	21	40	6	3	1563.43	22	1691.99	14.9	1691.71	1692.15
E022-06m	21	40	6	4	371.29	119.8	524.62	56.6	569.76	569.76
E023-03g	22	46	6	1	1507.88	185.9	3498.42	25	3867.49	3867.49
E023-03g	22	46	6	2	189.26	971	1373.65	53	1626.46	1646.73
E023-03g	22	46	6	3	273.44	766.8	1586.47	52.9	1974.97	1974.97
E023-03g	22	46	6	4	5.61	32930.3	1060.19	78	1441.26	1434.33
E023-05s	22	43	8	1	1557.51	155.3	3378.4	20.7	3603.53	3567.71
E023-05s	22	43	8	2	431.17	336.5	904.87	103.8	1498.52	1498.52
E023-05s	22	43	8	3	302.39	601.2	1525.76	48.1	1733.46	1733.46
E023-05s	22	43	8	4	103.63	1430.9	739.08	136.9	1205.51	1242.46
E026-08m	25	50	8	1	1377.23	244.1	4292.23	9.8	4330.84	4374.9
E026-08m	25	50	8	2	649.93	240.8	1764.06	25.2	1865.67	1873.13
E026-08m	25	50	8	3	112.19	2607.1	2572.51	13.7	2568.32	2585.9
E026-08m	25	50	8	4	241.93	366.1	721.96	56.2	782.02	804.12
E030-03g	29	62	10	1	498.09	1113.2	5055.43	19.6	5478.56	5513.29
E030-03g	29	62	10	2	152.13	1795.4	1123.26	162.2	2362.58	2382.67
E030-03g	29	62	10	3	31.25	12490.3	3196.52	23.6	3376.24	3414.38
E030-03g	29	62	10	4	0	inf	662.48	300.1	2061.7	2084.75
E030-04s	29	58	9	1	215.66	2492.4	4819.78	16	5098.99	5106.58
E030-04s	29	58	9	2	182.05	1343.9	1009.79	162.1	2153.64	2198.41
E030-04s	29	58	9	3	0	inf	2720.99	28.4	3040.34	3011.4
E030-04s	29	58	9	4	328.32	606	744.63	210.8	1818.46	1825.05
E031-09h	30	63	9	1	914.11	569.6	5515.13	9.6	5654.42	5690.16
E031-09h	30	63	9	2	346.84	733.9	2334.7	24.7	2510.29	2530.38
E031-09h	30	63	9	3	96.7	3606.3	3036.67	15.2	3101.13	3145.95
E031-09h	30	63	9	4	0	inf	468.51	142.2	770.42	776.31

Continued on next page

Table 7 – Continued from previous page

Instance	N	I	K	P	MILP		Matheuristic		Metaheuristics	
					Z_{MILP}	gap (%)	Z_{MH}	gap (%)	Z_{TS}^c	Z_{SA}^c
E033-03n	32	61	9	1	258.82	1936.9	545.27	879.7	3744.88	3801.92
E033-03n	32	61	9	2	42.54	5703	87.54	2645.8	1109.23	1109.23
E033-03n	32	61	9	3	0	inf	0	inf	886.78	644.5
E033-03n	32	61	9	4	270.96	2459.1	163.18	4104	5398.04	5504.99
E033-04g	32	72	11	1	93.37	7194.3	5220.6	30.5	5930.03	6023.92
E033-04g	32	72	11	2	162.45	1890.1	1864.58	73.4	2386.39	2408.21
E033-04g	32	72	11	3	0	inf	2538.04	74	3523.92	3635.05
E033-04g	32	72	11	4	131.06	3906.4	3128.29	67.8	4438.17	4466.66
E033-05s	32	68	10	1	509.99	1187	4735.32	34.9	5593.86	5619.94
E033-05s	32	68	10	2	139.79	2060.9	1722.75	75.3	2203.85	2255.95
E033-05s	32	68	10	3	226.21	1737.3	2674.43	55.4	3319.46	3403.26
E033-05s	32	68	10	4	333.07	1428.1	3580.23	38.3	4137.18	4180.3
E036-11h	35	63	11	1	892.11	571.2	5479.73	9.3	5528.19	5609.2
E036-11h	35	63	11	2	689.31	318.4	2319.06	22.3	2410.47	2427.42
E036-11h	35	63	11	3	160.32	2021.9	2946.35	19.1	2967.92	3021.2
E036-11h	35	63	11	4	0	inf	175.6	502.7	669.51	677.2
E041-14h	40	79	14	1	734.65	950.4	6785.79	13.5	6929.86	7048.28
E041-14h	40	79	14	2	614.28	508.3	2852.02	30.6	3044.73	3111.53
E041-14h	40	79	14	3	287.94	1483.2	3610.24	24	3785.3	3905.28
E041-14h	40	79	14	4	0	inf	125.28	1127.2	895.35	928.09
E045-04f	44	94	14	1	385.4	2322.8	4896.98	90.7	8192.69	8328.31
E045-04f	44	94	14	2	158.9	2832.8	2570.39	81.3	3663.21	3772.17
E045-04f	44	94	14	3	1	310202	1	309902.9	2031.81	2094.91
E045-04f	44	94	14	4	0	inf	1677.24	132.7	2845.41	2897.1
E051-05e	50	99	13	1	347.28	2696.4	6572.58	47.9	9182.85	9227.23
E051-05e	50	99	13	2	754.51	536.2	2618.66	83.2	4250.98	4308.42
E051-05e	50	99	13	3	59.94	8940.9	2539.95	113.7	4894.33	4943.42
E051-05e	50	99	13	4	0	inf	563.23	292.4	1711.59	1749.89
E072-04f	71	147	20	1	0	inf	5543.06	164	14124.06	14179.54
E072-04f	71	147	20	2	0	inf	2883.68	153.6	6802.26	6852.28
E072-04f	71	147	20	3	0	inf	3053.17	170.9	7758.66	7812.03
E072-04f	71	147	20	4	0	inf	282.89	642	1575.79	1641.39
E076-07s	75	155	18	1	277.23	5424.3	5289.77	189.5	14431.91	14528.48
E076-07s	75	155	18	2	384.38	1870.7	2468.53	206.9	6687.07	6791.3
E076-07s	75	155	18	3	100.32	8440.2	2347.75	265	7641.2	7790.38
E076-07s	75	155	18	4	0	inf	556.89	534.4	2610.68	2743.22
E076-08s	75	146	19	1	627.99	2194.3	4888.41	194.7	13459.14	13571.09
E076-08s	75	146	19	2	0	inf	2410.62	197.9	6238.92	6332.14
E076-08s	75	146	19	3	82.2	9931.4	2192.64	276	7299.2	7404.64
E076-08s	75	146	19	4	0	inf	172.52	1864.7	2451.36	2570.4
E076-10e	75	150	18	1	0	inf	4881.58	202.8	13889.45	14012.8
E076-10e	75	150	18	2	18	40528.3	2454.89	197.9	6448.15	6558.32
E076-10e	75	150	18	3	0	inf	1885.22	322.7	7127.36	7200.26
E076-10e	75	150	18	4	0	inf	365.17	791.9	2369.96	2481.92
E076-14s	75	143	18	1	87.83	15842.6	6128.31	128.5	13116.02	13245.51
E076-14s	75	143	18	2	0	inf	2376.44	191.6	6049.73	6173.32
E076-14s	75	143	18	3	2.83	270959.9	1955.25	292.7	6803.21	6920.43
E076-14s	75	143	18	4	0	inf	400.49	672.6	2239.36	2332.08
E101-08e	100	193	24	1	205.5	9163.1	3627.95	424.7	17943.79	18040.47
E101-08e	100	193	24	2	0	inf	1791.62	429.7	8345.24	8487.75
E101-08e	100	193	24	3	0	inf	1384.41	626.4	8671.26	8801.4
E101-08e	100	193	24	4	0	inf	512.74	805	3478.22	3664.84

Continued on next page

Table 7 – Continued from previous page

Instance	N	I	K	P	MILP		Matheuristic		Metaheuristics	
					Z_{MILP}	gap (%)	Z_{MH}	gap (%)	Z_{TS}^c	Z_{SA}^c
E101-10c	100	199	28	1	0	inf	3696.68	438.3	18303.81	18536.17
E101-10c	100	199	28	2	0	inf	1484.22	575.5	8472.14	8648.83
E101-10c	100	199	28	3	0	inf	1011.97	878.2	8254.63	8517.89
E101-10c	100	199	28	4	0	inf	468.94	1110.6	4100.26	4327.52
E101-14s	100	198	25	1	337.39	3440.3	4967.12	140.3	5818.94	5463.83
E101-14s	100	198	25	2	266.29	2495.7	2652.85	159.6	2825.7	2900.6
E101-14s	100	198	25	3	307.3	1929.7	2358.77	162.2	3028.63	3022.9
E101-14s	100	198	25	4	0.14	2088169.7	634.72	368.7	1253.06	1286
Average					488.4	inf	2262.43	3145	4174.16	4220.8

As shown in Table 7, none of the 108 MILP runs reached proven optimality within 1800 CPU seconds. Moreover, the MILP solver even failed to obtain a feasible incumbent solution in 28 cases. The MILP solver even failed to obtain a feasible incumbent solution in 28 cases and achieved the best solution only for instance E016-05m. In contrast, the proposed matheuristic consistently generated feasible solutions for all benchmark instances, achieving an average objective value more than four times higher than that obtained by the MILP solver.

Among the two metaheuristics, SA achieves the highest overall average objective value of 4220, compared with 4174 obtained by TS. SA outperforms TS on 85 of the 108 scenarios, while TS achieves better solutions in only eight cases. The advantage of SA is consistent across the first 16 instances listed in Table 7 (SA obtains better solutions in 43 of 64 cases). Its superiority also remains stable across all prize scenarios, as SA consistently achieves a higher average objective value without a systematic widening or narrowing of the performance gap as instance size increases. Although TS outperforms SA considerably less frequently, its improvements are larger on average when it does, allowing it to remain competitive on a small subset of instances. Overall, these results suggest that SA is the more robust approach for the 3DP-CPTP, as its performance advantage is largely independent of instance size and prize scenario.

The metaheuristics in Table 7 operate customer-based, enforcing that either every item of a customer is served by a single vehicle or none of its items is. This restriction has two opposing effects: it excludes solutions reachable by an item-based formulation, but it also reduces the search space that can be explored within a given time budget. Since the net effect of these two mechanisms cannot be determined analytically, we evaluate it empirically by implementing item-based variants of both metaheuristics, in which items rather than customers serve as the atomic units of all neighborhood moves.

The eight scenarios on instances E016-03m and E016-05m are small enough to be solved to proven optimality, requiring on average 8500 CPU seconds per instance, and thus permit a direct assessment of solution quality. Within 1800 CPU seconds, item-based SA (Z_{SA}^i) attains 99.97% of the optimal objective on average, matching the optimum in seven of the eight cases and deviating by 0.25% in the remaining one. The item-based TS (Z_{TS}^i) attains 99.04%, matching the optimum twice. Their customer-based counterparts reach only 96.40% (SA) and 96.39% (TS) and attain the optimum in none of the eight cases. On instances of this size, the item-based encoding is therefore clearly preferable, since its larger feasible set can still be searched thoroughly within the time budget.

Table 8: Comparison of the objective values obtained by customer-based TS (Z_{TS}^c , see Algorithm 2) and item-based TS (Z_{TS}^i), and customer-based SA (Z_{SA}^c , see Algorithm 3) and item-based SA (Z_{SA}^i), with the parameter configurations from Table 6 and a computation time of 1800 CPU seconds.

Instance	N	I	K	P	Metaheuristics			
					Z_{TS}^c	Z_{TS}^i	Z_{SA}^c	Z_{SA}^i
E016-03m-E016-05m	15	26-32	5	1-4	1288.04	1302.84	1287.91	1318.35
E021-04m-E026-08m	20-25	36-50	5-8	1-4	2063.67	2060.28	2068.58	2069.61
E030-03g-E041-14h	29-40	58-72	9-14	1-4	3322.81	3287.07	3358.17	3263.49
E045-04f-E076-07s	44-75	94-150	13-20	1-4	6639.16	6496.06	6730.82	6653.34
E101-08e-E101-14s	100	193-199	24-28	1-4	7541.30	7539.29	7641.52	7023.79
Average					4174.16	4126.46	4220.80	4106.54

As Table 8 shows, item-based decisions do not always lead to better SA and TS results. Averaged over all 108 scenarios, customer-based TS and SA outperform their item-based counterparts, and the item-based variants remain ahead only on instances with at most 25 customers and 50 items, by less than 0.4%. Most notably, for the largest instances, with 100 customers and approximately 200 items, customer-based SA produces solutions that are, on average, 8.8% better, whereas customer-based TS yields only a negligible improvement.

4.3. Robustness and Parameters Setting

A comprehensive parameter study was conducted across the 108 scenarios considered in Table 7.

Table 9 clarifies that the matheuristic (see Algorithm 1) is very sensitive to the number of iterations (I^{max}) and the time limit for each iteration (T_t). In particular, many short iterations (e.g., $(I^{max}, T_t) = (20, 10), (30, 10)$) lead to poor results. Across all 108 scenarios, the average objective values obtained under different parameter configurations range from 780 to 2262, demonstrating that appropriate parameter calibration can substantially improve solution quality in the matheuristic.

Table 9 Comparison of the objective values obtained by the matheuristic for different parameter settings with number of iterations $I^{max} = \{10, 20, 30\}$, time limits $T_t = \{10, 50, 100\}$ and stagnation thresholds

$\sigma_{MH}^{max} \in \{4, 5, 6\}$.				
I^{max}	T_t	$Z_{MH}(\sigma_{MH}^{max} = 4)$	$Z_{MH}(\sigma_{MH}^{max} = 5)$	$Z_{MH}(\sigma_{MH}^{max} = 6)$
10	10	804.97	791.90	779.88
10	50	1204.66	1264.87	1224.40
10	100	1424.68	1382.99	1392.22
20	10	923.29	953.64	883.58
20	50	1820.79	1858.39	1860.86
30	10	910.65	970.05	1195.63
30	50	2117.38	2262.43	2260.24

Table 10 Comparison of the objective values obtained by the tabu search for different parameter settings with tabu tenure $\theta \in \{5, 10, 20\}$, packing-feasibility check time $T_{TS}^c \in \{0.01, 0.1, 0.2\}$ and diversification thresholds

$\sigma_{TS}^{max} \in \{100, 300, 500\}$				
θ	T_{TS}^c	$Z_{TS}^c(\sigma_{TS}^{max} = 100)$	$Z_{TS}^c(\sigma_{TS}^{max} = 300)$	$Z_{TS}^c(\sigma_{TS}^{max} = 500)$
5	0.01	4103.32	4100.30	4098.16
5	0.1	4171.49	4170.98	4163.44
5	0.2	4156.78	4153.38	4146.62
10	0.01	4109.66	4107.15	4102.18
10	0.1	4171.78	4174.16	4168.06
10	0.2	4156.92	4163.98	4154.26
20	0.01	4099.28	4127.64	4091.08
20	0.1	4169.67	4169.89	4169.88
20	0.2	4158.84	4157.18	4158.21

However, the tabu search (see Algorithm 2) is much more robust across different parameter settings of the tabu tenure (θ), packing-feasibility check time (T_{TS}^c), and the diversification threshold (σ_{TS}^{max}).

Table 10 shows that the average objective value of the proposed TS varies only between 4091 and 4174 across the tested parameter configurations. This narrow variation in performance indicates that

TS is relatively insensitive to parameter settings and consistently produces high-quality solutions, demonstrating its robustness.

Similarly, SA (see Algorithm 3) exhibits robust performance across different parameter configurations, including variations in the cooling factor α , packing-feasibility check time T_{SA}^c , and reheat factor β .

Table 11 Comparison of the objective values obtained by SA for different parameter settings with cooling factor $\alpha \in \{0.999, 0.9999, 0.99999\}$, packing-feasibility check time $T_{SA}^c \in \{0.01, 0.05, 0.1\}$ and reheat factor $\beta \in \{0.01, 0.1, 0.3\}$

α	T_{SA}^c	$Z_{SA}^c (\beta = 0.01)$	$Z_{SA}^c (\beta = 0.1)$	$Z_{SA}^c (\beta = 0.3)$
0.999	0.01	4081.73	4127.95	4119.48
0.999	0.05	4188.52	4192.71	4188.73
0.999	0.1	4200.69	4204.07	4199.58
0.9999	0.01	4147.63	4145.07	4155.79
0.9999	0.05	4210.71	4220.80	4211.64
0.9999	0.1	4216.22	4216.40	4215.10
0.99999	0.01	4044.56	4090.98	4083.13
0.99999	0.05	3912.56	3896.88	3901.57
0.99999	0.1	3863.44	3866.14	3868.10

Table 11 clarifies that the average objective values of simulated annealing range from 3863 to 4220 across the tested parameter configurations on the 108 scenarios.

Since practical optimization problems are often solved under limited computational budgets, evaluating algorithm performance across different time limits is essential. Therefore, the results of the proposed matheuristic and metaheuristics are reported after 60, 180, 900, 1800, 3600, 7200, and 10800 CPU seconds of computation in Figure 8.

Figure 8 highlights substantial differences in the time-dependent performance of the three solution methods. Both metaheuristics rapidly identify high-quality solutions, considerably outperforming the matheuristic even under the shortest computational time budget of 60 seconds. SA does not show any substantial improvement after 900 CPU seconds.

Tabu search exhibits a similar pattern, although its improvement is slightly more gradual. Its average objective value increases substantially up to 900 seconds and continues to improve marginally as

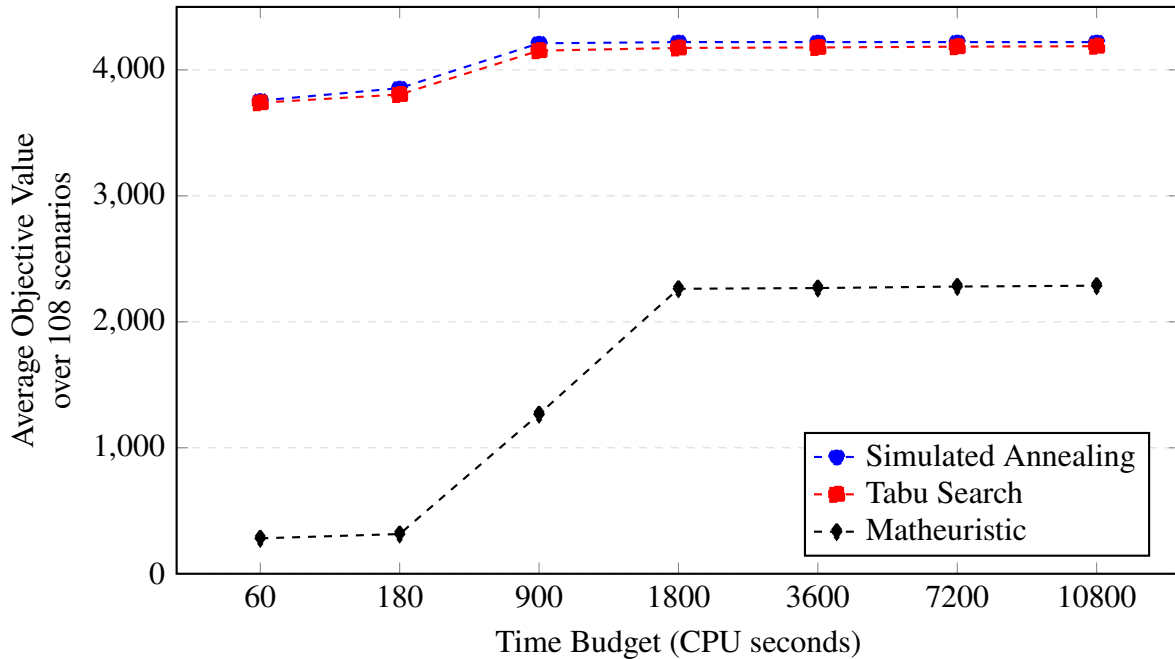


Figure 8 Objective value of the matheuristic (Algorithm 1), tabu search (Algorithm 2), and simulated annealing (Algorithm 3) over the 108 scenarios obtained under different computational time budgets after 60, 180, 900, 1800, 3600, 7200, and 10800 CPU seconds.

the computational budget is extended, reaching 4188 after 10800 CPU seconds, compared to 4174 after 1800 CPU seconds. Consequently, SA consistently outperforms TS for all considered time budgets, although the performance difference becomes comparatively small for longer runtimes.

In contrast, the matheuristic is considerably more sensitive to the available computational time. Its average objective value increases from 281 after 60 CPU seconds to 2262 after 1800 CPU seconds, with only marginal improvements thereafter. Nevertheless, even with the largest time budget of 10800 CPU seconds, the matheuristic remains substantially inferior to both metaheuristics operating with only 60 CPU seconds of computation time. Overall, the results indicate that the metaheuristics are substantially more effective at rapidly identifying high-quality solutions, while additional computational time provides only limited benefits once approximately 900 CPU seconds are available.

Our experimental results demonstrate that the MILP formulation and the matheuristic are insufficient for effectively solving 3DP-CPTP instances. Consequently, metaheuristic approaches are necessary to obtain high-quality solutions within reasonable time limits. Both TS and SA show strong robustness across the considered parameter settings, consistently achieving high-quality solutions across a wide range of configurations. We suggest that customer-based SA is the best choice for solving the 3DP-CPTP in general.

4.4. General Insights and Limitations

Building on the evaluation of the 3DP-CPTP in the previous sections, several findings provide insights that extend beyond the specific problem setting.

Item-based decisions are particularly relevant in medical drone logistics, where limited payload and cargo-bay capacity may prevent a single flight from serving a facility's entire request. A request comprising cold-chain containers of blood, vaccines, and diagnostic samples may exceed the capacity of a single vehicle but be accommodated across two flights. In such cases, serving the request requires individual items to be assigned to different vehicles.

The extensive computational experiments yield several insights into the computational difficulty of the 3DP-CPTP and closely related problem settings. Even instances with only 26 items and 5 vehicles cannot be solved to proven optimality within a reasonable computational time. Moreover, the matheuristic does not provide satisfactory solutions for instances with more than 40 items and 8 vehicles, making metaheuristic approaches such as TS and SA necessary for larger instances.

The results show that TS and SA substantially outperform both the MILP and the matheuristic, even when given only 1/30 of the computational time. This highlights their ability to identify high-quality solutions rapidly. SA achieves slightly higher average objective values than TS and obtains better solutions in approximately 80% of the cases. Under a fixed computational budget, restricting the metaheuristic search to customer-based decisions also consistently improves performance compared with allowing item-based decisions.

Among the algorithmic parameters considered, the packing-feasibility check has the clearest impact on the performance of both TS and SA, whereas the remaining parameter settings have comparatively little effect. The computational results further reveal an important relationship between instance size and the choice of decision representation. On small instances, item-based variants obtain near-optimal solutions and outperform their customer-based counterparts. As the instance size increases, however, this relationship reverses: restricting the search to customer-based decisions becomes advantageous, as the resulting smaller search space enables a more effective exploration within a given computational budget.

Despite these provided insights, our work has several limitations that should be considered when applying the proposed methodologies in real-world settings. The most significant limitation is that all requests are assumed to be known before optimization. In real-world operations, requests typically arrive sequentially, requiring immediate acceptance or rejection decisions under uncertainty about future requests. Another limitation is the assumption that request prizes are exogenously specified

by customers, whereas in many real-world settings, the value or revenue associated with serving a request is determined by the service provider rather than the customer. A further key limitation is that travel costs only regard distance and omit energy or battery constraints, as heavier loads reduce achievable range.

5. Conclusion

This paper investigates the *Three-Dimensional Packing Capacitated Profitable Tour Problem* (3DP-CPTP), a challenging optimization problem that requires the simultaneous selection of requests, determination of vehicle routes, and packing of items with different lengths, widths, and heights. We provide an overview of closely related contributions and existing solution approaches. The 3DP-CPTP is formulated as a mixed-integer linear program (MILP), and a matheuristic is developed that integrates MILP components into a heuristic search framework. In addition, two metaheuristics, namely *Tabu Search* (TS) and *Simulated Annealing* (SA), are developed to solve the 3DP-CPTP.

A key contribution of this study is the integration of profitable routing and three-dimensional packing within a single optimization framework, referred to as the 3DP-CPTP. The computational results indicate that neither the MILP nor the matheuristic provides satisfactory solutions within the considered computational limits, highlighting the need for more efficient metaheuristic approaches. Among the methods evaluated, SA with customer-based decisions achieved the best average results and is therefore recommended as the preferred solution approach for the 3DP-CPTP. The computational experiments further showed that the time allocated to the packing-feasibility check is the parameter with the most pronounced impact on the performance of both TS and SA. This finding highlights the importance of efficiently balancing the computational effort devoted to evaluating packing feasibility with the exploration of the overall solution space.

Future research could explore several different directions. One interesting direction is incorporating additional problem features, e.g., heterogeneous vehicle capacities that are common for different drone models. Another key extension involves making decisions online as requests arrive, with routing and packing finalized only once the booking horizon closes.

Appendix A: Diversify Function

The following Algorithm 4 describes the function `DIVERSIFY()` that is used in the tabu search (see Algorithm 2) and simulated annealing (see Algorithm 3) to explore new regions of the search space. The main notation used for Algorithm 4 is given in the following Table 12:

Diversification is a search mechanism designed to overcome stagnation by guiding the algorithm toward unexplored or less frequently visited regions of the solution space when no improvement is achieved over several consecutive iterations. Algorithm 4 describes how to diversify a solution of the 3DP-CPTP.

Table 12 Notation for the diversify function.

Symbol	Explanation
S	Incumbent solution
S^1	Set of accepted items in the incumbent solution S
S^0	Set of rejected items in the incumbent solution S
R^n	Random subset of accepted items that can be removed during diversification.
n^r	Number of accepted items removed in the diversification procedure (default 30%).
U	List of currently rejected items considered for reinsertion.
pos_{CI}	Best insertion index of an item according to the method CHEAPEST-INSERTION().
Δ_{CI}	Incremental routing cost induced by the method CHEAPEST-INSERTION().

Algorithm 4 Diversify Function

```

1: procedure DIVERSIFY( $S$ )
2:   if  $S^1 = \emptyset$  then
3:     return  $S$                                      ▷ Nothing to diversify
4:   end if
5:    $n^r \leftarrow \max(1, \lfloor |S^1|/3 \rfloor)$ 
6:    $R^n \leftarrow$  random subset of  $S^1$  of size  $n^r$ 
7:   for  $i^d \in R^n$  do                               ▷ Destroy items
8:     remove  $i^d$  from  $S^1$  to  $S^0$ 
9:   end for
10:   $U \leftarrow$  shuffled subset of  $S^0$  with  $|U| = 2n^r$ 
11:  for  $i^d \in U$  do                                   ▷ Try to reinsert items
12:     $k^* \leftarrow \text{None}$ ,  $pos_{CI}^* \leftarrow \text{None}$ ,  $\Delta^* \leftarrow \infty$ 
13:    for each incumbent route  $k$  do
14:       $(pos_{CI}, \Delta_{CI}) \leftarrow \text{CHEAPEST-INSERTION}(i^d, k)$ 
15:      if  $pos_{CI} \neq \text{None}$  and  $\Delta_{CI} < \Delta_{CI}^*$  then
16:         $k^* \leftarrow k$ ,  $pos_{CI}^* \leftarrow pos_{CI}$ ,  $\Delta_{CI}^* \leftarrow \Delta_{CI}$ 
17:      end if
18:    end for
19:    if  $k^* \neq \text{None}$  and  $p_{i^d} - \Delta_{CI}^* > 0$  then
20:      insert  $i^d$  at position  $pos_{CI}^*$  in route  $k^*$ 
21:      move  $i^d$  from  $S^0$  to  $S^1$ 
22:    end if
23:  end for
24:  return  $S$ 
25: end procedure

```

The diversification procedure in Algorithm 4 partially destroys the current solution S by randomly removing a subset of selected items R^n from the solution and moving them to the unselected set U . The removed items are then considered for reinsertion in a randomized order using a cheapest-insertion heuristic, where the best feasible insertion position across all routes is identified. An item is reinserted only if the resulting profit improvement remains positive (line 19). This process perturbs the current solution S while preserving promising structures, allowing the search to explore new regions of the solution space.

References

- Adwibowo A, 2021 *Assessments of heavy lift uav quadcopter drone to support covid 19 vaccine cold chain delivery for indigenous people in remote areas in south east asia*. *MedRxiv* 2021–01.
- Angelelli E, Archetti C, Filippi C, Vindigni M, 2021 *A dynamic and probabilistic orienteering problem*. *Computers & Operations Research* 136:105454.

- Archetti C, Feillet D, Hertz A, Speranza MG, 2009 *The capacitated team orienteering and profitable tour problems. Journal of the Operational Research Society* 60(6):831–842.
- Bortfeldt A, Gehring H, 2006 *A tabu search algorithm for the container loading problem. European Journal of Operational Research* 169(3):747–766.
- Bortfeldt A, Yi J, 2018 *The split delivery vehicle routing problem with three-dimensional loading constraints. Working Paper Series* .
- Bortfeldt A, Yi J, 2020 *The split delivery vehicle routing problem with three-dimensional loading constraints. European Journal of Operational Research* 282(2):545–558.
- Cabrera-Guerrero G, Lagos C, Castañeda C, Johnson F, Paredes F, Cabrera E, 2017 *Parameter tuning for local-search-based matheuristic methods. Complexity* 2017: 1702506(1).
- Caspar M, 2026 *A matheuristic for the split delivery capacitated profitable tour problem with incomplete service. Central European Journal of Operations Research* 34(1):211–238.
- Caspar M, Schermer D, Wendt O, 2023 *Formulations for the split delivery capacitated profitable tour problem. International Conference on Computational Science and Its Applications*, 82–98 (Springer).
- Cen X, Zhou G, Ji B, Yu SS, Zhang Z, Fang X, 2023 *Modelling and heuristically solving three-dimensional loading constrained vehicle routing problem with cross-docking. Advanced Engineering Informatics* 57:102029.
- Chi J, He S, Song R, 2024 *Solving capacitated vehicle routing problem with three-dimensional loading and relocation constraints. Computers & Operations Research* 173:106864.
- Clarke G, Wright JW, 1964 *Scheduling of vehicles from a central depot to a number of delivery points. Operations research* 12(4):568–581.
- Crainic TG, Perboli G, Tadei R, 2008 *Extreme point-based heuristics for three-dimensional bin packing. INFORMS Journal on Computing* 20(3):368–384.
- Feillet D, Dejax P, Gendreau M, 2005 *Traveling salesman problems with profits. Transportation science* 39(2):188–205.
- Feng Y, Hu T, Chen XA, Wang G, 2025 *Recent advances in knapsack problem: A comprehensive review of models, algorithms, and applications. Neurocomputing* URL <http://dx.doi.org/10.1016/j.neucom.2025.131611>.
- Ferreira KM, Munari P, Alves de Queiroz T, Archetti C, Morabito R, 2025 *Exact methods for the split delivery vehicle routing problem with two-dimensional loading constraints. Networks* 86(4):361–388.
- Fischetti M, González JJS, Toth P, 1998 *Solving the Orienteering Problem through Branch-and-Cut. INFORMS Journal on Computing* 10(2):133–148, URL <http://dx.doi.org/10.1287/ijoc.10.2.133>, publisher: INFORMS.
- Gendreau M, Iori M, Laporte G, Martello S, 2006 *A tabu search algorithm for a routing and container loading problem. Transportation Science* 40(3):342–350, URL <http://www.jstor.org/stable/25769310>.

- Gilmore PC, Gomory RE, 1965 *Multistage cutting stock problems of two and more dimensions*. *Operations Research* 13(1):94–120, URL <http://dx.doi.org/10.1287/opre.13.1.94>.
- Glover F, 1986 *Future paths for integer programming and links to artificial intelligence*. *Computers & Operations Research* 13(5):533–549, URL [http://dx.doi.org/10.1016/0305-0548\(86\)90048-1](http://dx.doi.org/10.1016/0305-0548(86)90048-1).
- Höhn M, 2026 *Drohnenlieferung: Von der Apotheke zum Patienten*. *Pharmazeutische Zeitung* URL <https://www.pharmazeutische-zeitung.de/drohnenlieferung-von-der-apotheke-zum-patienten-162646/>.
- Jackson WG, Özcan E, John RI, 2017 *Tuning a simulated annealing metaheuristic for cross-domain search*. *2017 IEEE congress on evolutionary computation (CEC)*, 1055–1062 (IEEE).
- Jeon HH, Lucarelli C, Mazarati JB, Ngabo D, Song H, 2022 *Last-mile delivery in health care: drone delivery for blood products in rwanda*. Available at SSRN 4214918 .
- Kacprzak L, Rudy J, Żelazny D, 2015 *Multi-criteria 3-dimension bin packing problem*. *Research in Logistics & Production* 5.
- Kirkpatrick S, Gelatt Jr CD, Vecchi MP, 1983 *Optimization by simulated annealing*. *science* 220(4598):671–680.
- Ma Y, Zhou Y, Fang Q, Xia S, Chen W, 2025 *A three-dimensional container loading algorithm for solving logistics packing problem*. *EURO Journal on Transportation and Logistics* 100167.
- Moura A, Pinto T, Alves C, Valério de Carvalho J, 2023 *A matheuristic approach to the integration of three-dimensional bin packing problem and vehicle routing problem with simultaneous delivery and pickup*. *Mathematics* 11(3):713.
- Nascimento DN, Toledo FMB, Coelho LC, 2026 *Logic-based benders decomposition for the three-dimensional loading capacitated vehicle routing problem with time windows*. *European Journal of Operational Research* .
- Nirwan S, Nirwan NF, 2025 *Advancing vehicle routing with three-dimensional load constraints: A novel genetic algorithm and space optimization approach*. *Internet of Things and Artificial Intelligence Journal* 5(3):760–777.
- Perić N, Kolak A, Lešić V, 2025 *Modular coordination of vehicle routing and bin packing problems in last mile logistics*. *Logistics* 9(2):70.
- Rojas-Saavedra JE, Álvarez-Martínez D, Escobar JW, 2026 *Boosting sustainable development goals: a hybrid metaheuristic approach for the heterogeneous vehicle routing problem with three-dimensional packing constraints and fuel consumption*. *Annals of Operations Research* 358(1):401–446.
- Schermer D, Moeini M, Wendt O, 2019 *A matheuristic for the vehicle routing problem with drones and its variants*. *Transportation Research Part C: Emerging Technologies* 106:166–204.
- Schoepf S, Mak S, Senoner J, Xu L, Netland T, Brintrup A, 2025 *Towards scalable three-dimensional loading capacitated vehicle routing*. *IFAC-PapersOnLine* 59(10):2939–2944.
- Tamke F, Linß F, Kuttner L, Buscher U, 2025 *A branch-and-cut algorithm for vehicle routing problems with three-dimensional loading constraints*. *European Journal of Operational Research* .

- Vu THG, Nong QK, 2026 *A feasibility-driven anytime hybrid search for the 3l-cvrp with com-aware load balancing. IEEE Access* .
- Wäscher G, Haußner H, Schumann H, 2007 *An improved typology of cutting and packing problems. European Journal of Operational Research* 183(3):1109–1130, URL <http://dx.doi.org/10.1016/j.ejor.2005.12.047>.
- Xu J, Chiu SY, Glover F, 1998 *Fine-tuning a tabu search algorithm with statistical tests. International Transactions in Operational Research* 5(3):233–244.
- Yin X, Yu Z, Liu Y, Chen Y, Guo A, 2025 *An improved approach for vehicle routing problem with three-dimensional loading constraints based on genetic algorithm and residual space optimized strategy. Processes* 13(5):1449.
- Yu VF, Susanto H, Jodiawan P, Ho TW, Lin SW, Huang YT, 2022 *A simulated annealing algorithm for the vehicle routing problem with parcel lockers. IEEE Access* 10:20764–20782, URL <http://dx.doi.org/10.1109/ACCESS.2022.3152062>.
- Zharinova S, 2026 *Die Drohne als Medikamentenbote? Das ist bereits möglich. Apotheken Umschau* URL <https://www.apotheken-umschau.de/gesundheitspolitik/drohne-als-medikamentenbote-welches-potential-fachleute-sehen-1424043.html>.